

Decorators & functional patterns

Wrap functions to add behavior without touching their code.

Two weeks down. Opening Week 3.

Weeks 1-2 are done. OOP toolkit and pygame patterns are now stable. Week 3 shifts focus: Python stops living in the script and starts driving the rest of the machine.

✓ **DONE · WEEKS 1 - 2**

OOP + pygame

- Vector3D — operator overloading, @property
- Shape / Circle / Box — validation, factories
- Custom exceptions, hardened sandbox loop
- Spring-Mass-Damper (matplotlib + pygame)
- Sprite groups, collisions, sound, scenes
- Pong with AI paddle, brick-breaker stretch

→ **THIS WEEK · WEEK 3**

Python drives your laptop

- Day 15 — decorators + functional patterns
- Day 16 — subprocess + osascript bridge
- Day 17 — pyautogui + global hotkeys
- Day 18 — autoclicker build
- Day 19 — screen recognition, window control
- Day 20-21 — Morning Launcher capstone

Why decorators first? Hotkey bindings, retry-on-failure, timing — every Week 3 tool repeats the same wrapper shape. Learn it once.

A function that wraps another.

A decorator takes a function as input, builds a new function that does extra work around it, and returns the new one. The original still runs — just inside a wrapper.

```
def loud(fn):  
    def wrapper():  
        print(">>> calling")  
        result = fn()  
        print(">>> done")  
        return result  
    return wrapper  
  
def greet():  
    return "hi"  
  
greet = loud(greet)  # wrap it  
greet()
```

OUTPUT

```
>>> calling  
>>> done
```

Anatomy

loud(fn)

Outer function. Takes any function as argument.

wrapper()

Inner function. Runs extra logic, calls fn(), returns its result.

return wrapper

Outer hands back the inner — fn is now replaced by wrapped version.



TAKEAWAY

Original function isn't modified. A new function takes its name.

Same thing, less typing.

The @ symbol above a function definition tells Python to immediately pass that function through the named decorator. It's pure syntactic sugar — there's no new behavior here.

WITHOUT @

```
def greet():  
    return "hi"  
  
greet = loud(greet)  
  
greet()
```

WITH @

```
@loud  
def greet():  
    return "hi"  
  
greet()
```

@loud above def greet ≡ greet = loud(greet) — Python runs the rebind for you, the moment greet is defined.

NOTE

Read @name as 'pass this function through name before anyone else gets it'.

Wrappers must accept any signature.

*args collects positional arguments into a tuple. **kwargs collects keyword arguments into a dict. Together they let one wrapper handle functions of any shape — no rewriting per case.

```
def loud(fn):
    def wrapper(*args, **kwargs):
        print(">>>", args, kwargs)
        return fn(*args, **kwargs)
    return wrapper

@loud
def add(a, b): return a + b

@loud
def greet(name, loud=False):
    msg = f"hi {name}"
    return msg.upper() if loud else msg

add(2, 3)
greet("yuvraj", loud=True)
```

How they work

*args

Any number of positional args. Collected as a tuple. Empty if none passed.

**kwargs

Any number of keyword args. Collected as a dict. Empty if none passed.

fn(*args, **kwargs)

Unpacks them back when calling the original — args become positional again, kwargs become keyword.

OUTPUT

```
>>> (2, 3) {} >>> ('yuvraj',) {'loud': True}
```

Don't lose the original's identity.

When a decorator wraps a function, the new function's `__name__` and docstring become those of the inner wrapper. `functools.wraps` copies the original's metadata back, so debugging and `help()` still show what you wrote.

BEFORE · metadata lost

```
def loud(fn):
    def wrapper(*a, **k): ...
    return wrapper

@loud
def greet(): """says hi"""

greet.__name__
```

OUTPUT

'wrapper' ← wrong!

AFTER · metadata preserved

```
from functools import wraps

def loud(fn):
    @wraps(fn)
    def wrapper(*a, **k): ...
    return wrapper

greet.__name__
```

OUTPUT

'greet' ← correct

TAKEAWAY

Always `@wraps(fn)` inside your decorator. Costs one line, saves hours of debugging.

@retry(3) — one more layer.

A plain decorator takes one function and returns one. To pass settings (like a retry count), you write a function that builds and returns the decorator. Three nested levels — outer factory, middle decorator, inner wrapper.

```
def retry(n):                # outer: takes setting
    def decorator(fn):      # middle: takes function
        @wraps(fn)
        def wrapper(*a, **k): # inner: takes call args
            for attempt in range(n):
                try: return fn(*a, **k)
                except Exception as e: last = e
            raise last
        return wrapper
    return decorator

@retry(3)                    # retry(3) RUNS, returns decorator
def fetch(url): ...
```

Three layers

retry(n)

Takes the setting. Returns a decorator that knows about n.

decorator(fn)

Takes the target function. Returns a wrapper that knows about both n and fn.

wrapper(*a, **k)

Runs at every call. Sees args, knows n and fn from the closure.

💡 TAKEAWAY

If your decorator needs settings, it's no longer a decorator — it's a function that builds one.

Tiny functional helpers.

lambda makes a one-line anonymous function. map applies a function to every item in a sequence. filter keeps only the items where a function returns True. Comprehensions usually win for readability — these are for short, inline cases.

1. lambda — inline mini-function

```
square = lambda x: x ** 2  
square(5)           # 25
```

2. map — apply to every item

```
nums = [1, 2, 3, 4]  
squared = list(map(square, nums))  
# [1, 4, 9, 16]
```

3. filter — keep where True

```
evens = list(filter(lambda x: x % 2 == 0, nums))  
# [2, 4]
```

When to reach for them

lambda

Use only when the body is one short expression and you pass it inline. If you'd name it, use def.

map / filter

Fine for one-liners. For anything longer, a list comprehension reads better.

In Week 3

You'll see lambdas as click callbacks and hotkey handlers — short, inline, no name needed.

NOTE

`[x**2 for x in nums]` is usually clearer than `map(lambda x: x**2, nums)`. Use comprehensions by default.

decorators_lab.py

Build three decorators. Apply them to dummy functions. Confirm each does what its name claims.

@timer

Measures execution time.

- Print `fn.__name__` + elapsed ms.
- Use `time.perf_counter()`.
- Return whatever fn returned.
- `@wraps(fn)` on the wrapper.

@retry(n)

Reruns on failure.

- Take `n` as outer arg.
- Catch Exception, retry up to `n` times.
- If all fail, re-raise the last one.
- Test with a fn that fails twice then works.

@log_calls

Records inputs and output.

- Print fn name + args + kwargs.
- Call fn, capture result.
- Print return value, return it.
- Stack with `@timer` — order matters.

Stretch · stack @timer above @log_calls, swap the order, observe which prints first.

Things that quietly break.

 WATCH OUT

Forgetting return

If your wrapper forgets to return `fn(*a, **k)`, every decorated function silently returns `None`. Run it once and check the return value before moving on.

 WATCH OUT

Stacking order

Decorators apply bottom-up. `@timer` on top of `@log_calls` means timer wraps the already-logged function. Reading order: closest to `def` runs first at call time.

 TIP

When NOT to decorate

If the wrapper logic is needed in only one place, skip the decorator and inline it. Decorators earn their weight when the same wrap appears 3+ times.

 TIP

Closures do the work

Inner wrappers remember variables from the outer scope (closures). That's how `@retry(3)` keeps `n` around without passing it explicitly. Whole pattern leans on this.

The decorator shape repeats all week.

DAY 17

Global hotkeys

@hotkey('cmd+shift+1') binds a function to a key combo. Same shape: register-on-decoration.

DAY 18

Autoclicker

Toggle on/off via decorated hotkey handlers. Cleaner than a long if-elif on key event.

DAY 19+

Retry + timing

Screen recognition occasionally misses. @retry(3) and @timer earn their keep on flaky I/O.

Tomorrow: subprocess + osascript bridge — Python opens apps, runs shell commands, fires AppleScript one-liners.